

論理回路と論理演算

2026 年度高 2 情報

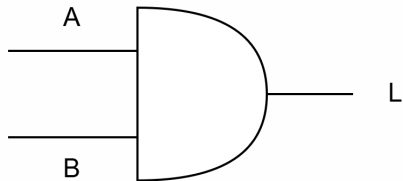
2026/05/16

机に貼ってある番号に従って着席・課題を出す方は前の机に
PC の準備は不要です

論理演算 (p76)

- コンピュータでは0と1を用いて演算(及び処理)をしている
- この演算(及び処理)を**論理回路**という
- あるスイッチの入力と出力の関係を表す表を**真理値表**という
- 論理回路の設計図を**回路図**という

AND回路 (p76)



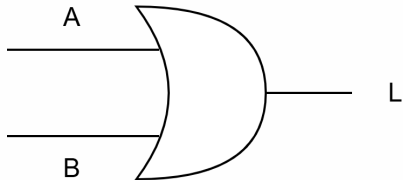
AND回路の図記号

数学の「かつ」と同じ動きをする

入力		出力
A	B	L
0	0	0
0	1	0
1	0	0
1	1	1

AND回路の真理値表

OR回路 (p77)



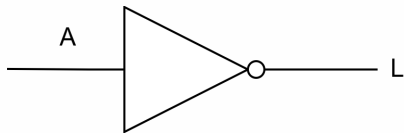
OR回路の図記号

数学の「または」と同じ動きをする

入力		出力
A	B	L
0	0	0
0	1	1
1	0	1
1	1	1

OR回路の真理値表

NOT 回路 (p77)



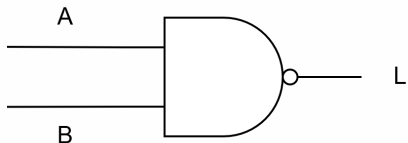
NOT 回路の図記号

数学の否定と同じ動きをする

入力	出力
A	L
0	1
1	0

NOT 回路の真理値表

NAND回路 (p76)



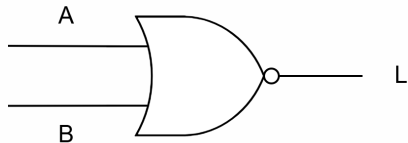
NAND回路の図記号

入力		出力
A	B	L
0	0	1
0	1	1
1	0	1
1	1	0

NAND回路の真理値表

数学の「かつ」の否定と同じ動きをする

NOR回路 (p76)



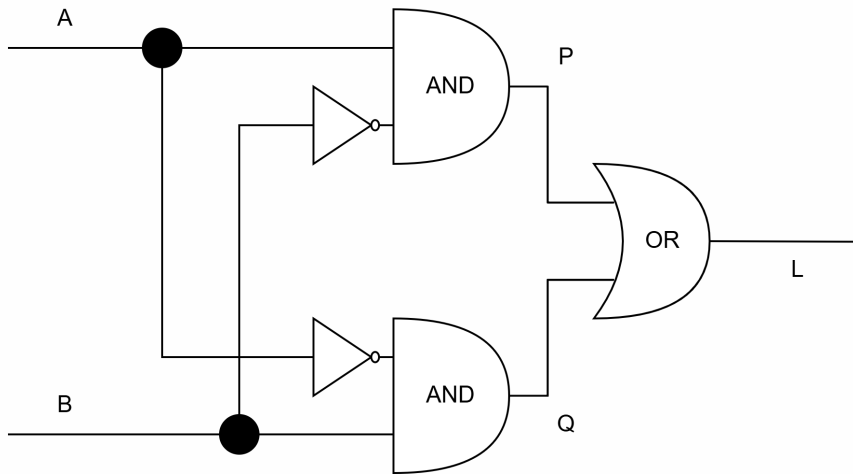
NOR回路の図記号

入力		出力
A	B	L
0	0	1
0	1	0
1	0	0
1	1	0

NOR回路の真理値表

数学の「または」の否定と同じ動きをする

XOR回路 (p76)



黒丸は分岐点. 黒丸が無い場所は分岐ではない.

XOR回路の真理値表

入力			出力
A	B	P Q	L
0	0		
0	1		
1	0		
1	1		

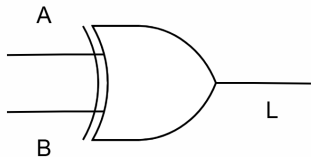
XOR 回路の真理値表 (p76)

入力				出力
A	B	P	Q	L
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

XOR回路について (p76)

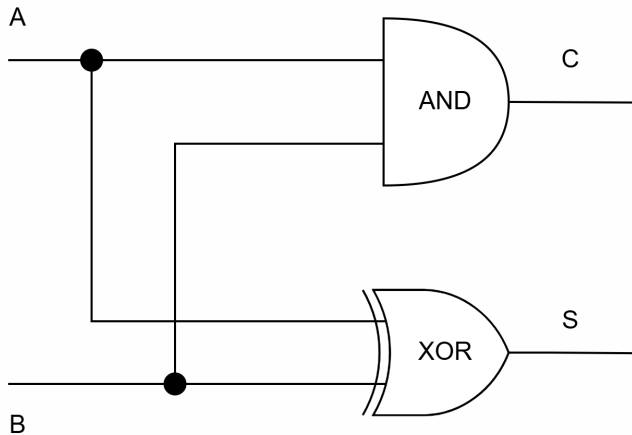
XOR回路の出力に注目しよう

- AかBどちらか一方が1のときだけ1を出力している
 - AもBも0のときは0を出力
 - AもBも1のときは0を出力
- ⇒ 2進数の足し算を表現する回路として使えそう
- XOR回路のことを**排他的論理和回路**ともいう.



XOR回路の図記号

半加算器 (p78)



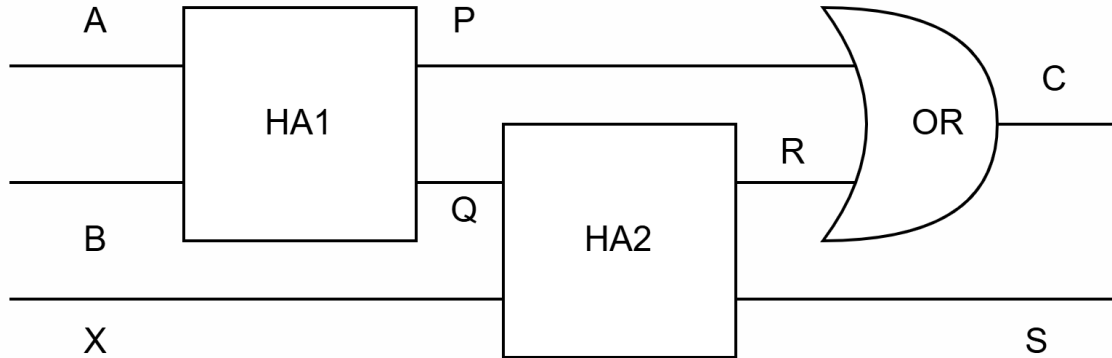
半加算器の出力 (p78)

入力		出力	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

半加算器の真理値表

- 1桁どうしの2進数の足し算はこれで大丈夫そう
- 2桁以上の計算はできるか？
- 繰り上がりを考慮できていない
- 繰り上がりを考慮できるような回路を考えてみよう.

全加器 (p79)



全加算器 (p79)

入力						出力	
A	B	X	P	Q	R	C	S
0	0	0					
0	1	0					
1	0	0					
1	1	0					
0	0	1					
0	1	1					
1	0	1					
1	1	1					

全加算器の真理値表

全加算器 (p79)

入力						出力	
A	B	X	P	Q	R	C	S
0	0	0	0	0			
0	1	0	0	1			
1	0	0	0	1			
1	1	0	1	0			
0	0	1	0	0			
0	1	1	0	1			
1	0	1	0	1			
1	1	1	1	0			

全加算器の真理値表

- (1) 半加算器1つ目
(HA1) を処理
($A+B$ を P, Q に
反映)

全加算器 (p79)

入力						出力	
A	B	X	P	Q	R	C	S
0	0	0	0	0	0		0
0	1	0	0	1	0		1
1	0	0	0	1	0		1
1	1	0	1	0	0		0
0	0	1	0	0	0		1
0	1	1	0	1	1		0
1	0	1	0	1	1		0
1	1	1	1	0	0		1

全加算器の真理値表

- (1) 半加算器1つ目 (HA1) を処理
($A+B$ を P, Q に反映)
- (2) 半加算器2つ目 (HA2) を処理
($Q+X$ を R, S に反映)

全加算器 (p79)

入力						出力	
A	B	X	P	Q	R	C	S
0	0	0	0	0	0	0	0
0	1	0	0	1	0	0	1
1	0	0	0	1	0	0	1
1	1	0	1	0	0	1	0
0	0	1	0	0	0	0	1
0	1	1	0	1	1	1	0
1	0	1	0	1	1	1	0
1	1	1	1	0	0	1	1

全加算器の真理値表

- (1) 半加算器1つ目 (HA1) を処理
($A+B$ を P, Q に反映)
- (2) 半加算器2つ目 (HA2) を処理
($Q+X$ を R, S に反映)
- (3) OR 回路を処理
(P または R を C に反映)

全加算器について (p79)

回路について

- $A+B$ に桁上がり X を考慮したもの
- 一般に n bit の演算には n 個の全加算器が必要 (または, $n - 1$ 個の全加算器と 1 つの半加算器)

論理演算 (p79)

真 (1) か偽 (0) になる演算のことを論理演算という.

- **論理積**

2つの値が両方 1 ならば 1 を返す

例：1100 と 0110 の論理積は 0100

- **論理和**

2つの値のうちどちらかが 1 なら 1 を返す

例：1100 と 0110 の論理和は 1110

- **否定**

値を反転させる (0 なら 1 を, 1 なら 0 を返す)

例：1100 の否定は 0011

論理演算の例 (p79)

ビットセット

特定の bit のみを強制的に 1 にする操作のことを**ビットセット**という。
ビットセットは論理和を用いることでうまくいく。

例) 8bit の数の 5bit 目を 1 にしたいとき, 00010000 との論理和を取る

ビットクリア

特定の bit のみを強制的に 0 にする操作のことを**ビットクリア**という。
ビットクリアは論理積を用いることでうまくいく。

例) 8bit の数の 5bit 目を 0 にしたいとき, 11101111 との論理積を取る

論理演算の例 (p79)

ビット反転

特定の bit のみを強制的に反転させる操作のことを**ビット反転**という。
ビット反転は排他的論理和を用いることでうまくいく。

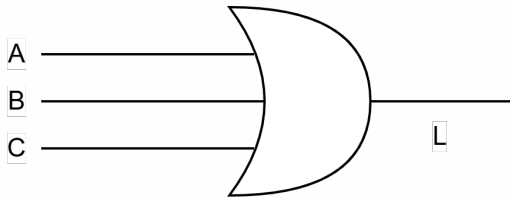
例) 8bit の数の 5bit 目を反転したいとき, 00010000 との排他的論理和を取る

特定の bit の取り出し

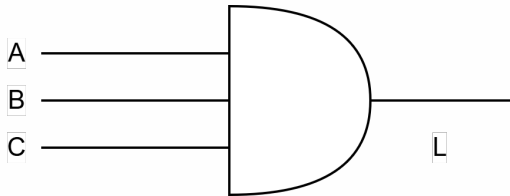
論理積を利用することで実現できる。

例) 8bit の数の下位 4bit を取り出すためには, 00001111 との論理積をとる。

課題の注意



これは3つの論理和
A, B, Cのいずれかが1なら1
A, B, Cの全て0なら0



これは3つの論理積
A, B, Cのすべてが1なら1
A, B, Cのいずれかが0なら0

略記のためにこれらを用いている.